

作业 5

——亚线性时间估算分位点——

声明：题目来源于 <http://openjudge.cn/>，解题思路参考教授上课讲解内容，代码实现由本人完成。分享这个文档仅限于学术交流，不用于任何形式的商业用途。

1. 题目

描述

有 n 个互不相同的数 $a_1, \dots, a_n$ ，你只能通过给出的函数接口来访问某一编号 i 的数的大小 a_i ，并返回你估计的这 n 个数的 p 分位数， $0 \leq p \leq 1$ ，0 分位数表示最小值，1 分位数表示最大值。

具体地，设你输出的数在 n 个数中从小到大的排名为 r ，要求 $r \in [(p - e)n, (p + e)n]$, $e = 0.03$

以交互题的形式提交，你需要实现一个函数 `rnk (n, p)`，返回估计的 p 分位数，唯一访问 a 的方式是利用 `query (i)` 来得到 a_i ，代码需包含头文件 `rank.h`。

$10^5 \leq n \leq 10^6$ ，要求访问次数不超过 5000。

输入

第一行一个正整数 n 和一个实数 p 。

第二行 n 个 `int` 范围内的整数，表示这 n 个互不相同的数 a_1 到 a_n 。

注意：这里描述的是链接后的完整程序的输入/输出格式，仅供测试用途。你提交的程序只能通过 `query` 函数访问数据，请勿自行读入！

输出

无

样例输入

```
5 0.5
1 2 3 4 5
```

样例输出

```
your answer is : 3
the times you sample is : 5
the rank of your answer is : 3
correct!
```

提示

在以下链接下载交互库 `grader.cpp`，`rank.h`，以及样例代码 `rank_sample.cpp`：

<https://github.com/crasysky/rank/tree/main>

另外，本题只考虑 n 较大的情况，样例仅供参考输入输出格式。

2. 思路

这道题目已经把思路说得非常详细了。我们主要说说为什么需要这样做。传统上，我们可以对所有数据进行排序之后寻找 p 分位数，但是排序的时间复杂度是 $O(n \log n)$ 。这需要存储访问整个数据集。然而，如果这些数据分布式存储在非常多的机器硬盘上，无法载入内存，或者开销太大，我们能否找到一个近似解答呢？¹

根据题目的意思，对于所有数据，我们不能全部获取。我们只能选择性地访问其中的至多5000个数据。这类似于“抽样调查”，通过抽样的结果来获得总体的情况。我们可以随机地选择5000个数据进行访问，然后对这些数据进行排序，找到其中的 p 分位数。由于这些数据是随机抽取的，因此它们的分布应该与总体数据的分布相似，因此我们得到的 p 分位数应该也是一个近似解。

接下来，我们从数学上论证这样做法的精确程度：

为了简化问题，我们考虑 $p = 0.5$ 的情况，也就是中位数。我们希望输出的数据在 $(0.5 - 2\varepsilon, 0.5 + 2\varepsilon)$ 范围内。设 $\alpha = 0.5 - 2\varepsilon, \beta = 0.5 + 2\varepsilon$ ，考虑子集 A_1, A_2, A_3 分别代表排序后位次是 $[1, \alpha n], [\alpha n + 1, \beta n], [\beta n + 1, n]$ 的数。对于一个平均采样，各有 $0.5 - 2\varepsilon$ 的概率落在 A_1, A_3 中。

这里需要 Chernoff Bound 不等式：²

$$\Pr\left(|\bar{X} - \mu| \geq \sqrt{\frac{\log(\frac{2}{\delta})}{2n}}\right) \leq \delta$$

其中， $\bar{X}$ 是样本的均值， μ 是它的期望值， n 是采样的次数， δ 是允许的失败概率。

在这里，我们设定 $\delta = 2\varepsilon$ ，式子变成

$$\Pr\left(|\bar{X} - \mu| \geq \sqrt{\frac{\log(\frac{1}{\varepsilon})}{2n}}\right) \leq 2\varepsilon$$

根据题目， $\varepsilon = 0.015$ ，因此， $\sqrt{\frac{\log(\frac{1}{\varepsilon})}{2n}} < 0.015$ ，当 $n > 2334$ 时，这个不等式成立。因此，采样5000次是足够的。

3. 代码实现

```
#include <bits/stdc++.h>
#include "rank.h"
using namespace std;
int rnk(int n, double p){
    minstd_rand gen;
    int T = 5000;
    vector<int> v;
    for (int i = 1; i <= T; i++)
    {
        int idx = gen() % n + 1;
        v.push_back(query(idx));
    }
    sort(v.begin(), v.end());
    return v[max(int(p * T) - 1, 0)];
}
```

¹曾经我们做的题目都是要求输出一个精确的解答，但是在实际生活中，由于采样的数据可能都是含有很多不可避免的噪声的，因此，我们得到的所谓的精确的解答并不一定是真正精确的。与此同时，我们还花费了很多资源在求解这样的“理论上精确解”，这并不是很好的，因此，很多时候，我们只需要得到一个近似解，并且保证它的精度不会太差就可以了。

²这里的log是指以 e 为底的对数。并且要求 $X_i \in [0, 1]$ 。对于更大的数据，可以通过数学手段映射到 $[0, 1]$ 区间内。